Shortest Normal Distance Between Convex Hulls

An iterative method is presented to compute the interaction between two 3-dimensional convex hulls i and j, which are arbitrarily translated and rotated in space. When the objects are in or near contact, it is necessary to locate the center of their (potential) contact and compute the minimum normal distance (separation or penetration) between the two surfaces. Each convex hull surface has two local degrees of freedom, so the problem requires iteratively solving for four unknowns. A GJK algorithm is used to quickly zero in on the (potential) contact region, and then a Newton-Raphson iteration (NRi) method, using a 4 by 4 positive-definite matrix, accurately zooms in on the contact region. The goal of this development is to present a rapid method for detecting (potential) contact, zeroing in on the (potential) contact point, and accurately computing surface penetration and contact forces.

The solution algorithms presented here depend on *rapidly* locating the extreme face of a convex hull when given a direction vector. If the surface has been discretized into a set of faces, a hash table may be used to quickly locate the face whose outward-facing normal vector most closely aligns with the direction vector. This allows a GJK algorithm to rapidly detect (potential) contact, and then NRi to zoom in on the region for accurate object interaction modeling.

To avoid problems generating a generating a hash-table loop-up for each convex hull, the following modification is proposed. In each convex hull file, merge into a single polygon face all adjacent triangles common to each flat region, and associate a single face normal with it. For the GJK algorithm any vertex common to the face may be returned, because they all have the same maximum extent in the direction of the face normal.

The purpose of this work is to develop procedures to rapidly compute representative interaction between any two arbitrary convex hulls, so it is assumed, worst-case, that both surfaces have been discretized and are represented by a finite number of vertices and faces. When the surfaces are represented analytically, special functions must be supplied to return the same information required from the general discretized method. The following discussion summarizes what is required for the GJK and NRi algorithms.

The GJK search algorithm demands are quite modest: for a given convex hull provide a direction vector and receive back an extreme vertex point on the face whose outward-facing normal vector most closely aligns with the direction vector. To speed up the process of locating the face, a preprocessor is first used to generate a set of uniformly spaced normal vectors on a unit sphere. Two parametric variables, $0 \le \phi \le \pi$ and $0 \le \theta \le 2\pi$, are used to represent the entire surface of the sphere, so there will be a unique ϕ and θ pair for each normal vector. Each unique ϕ and θ pair is used to generate a unique hash-table keyword, which is associated

with a unique index to a face of the convex hull. Then for any direction vector, values for ϕ and θ may be computed and converted into one of the unique keywords, which is inserted into the hash table to obtain the index of the corresponding face of the convex hull. A few small issues making this method work will be discussed later.

The NRi algorithm is a bit more demanding. It requires a 4 by 4 positive definite matrix to iteratively solve for four variables ϕ_i and θ_i for convex hull i, and ϕ_j and θ_j for convex hull j. The constraint equations for this problem require that the minimum-distance normal vector from surface i to surface j be parallel to both normal vectors. Let p and q represent the respective nearest points on surfaces i and j; let $\vec{r}_{pq}$ represent the vector from point p to point q and the desired normal vector to the two surfaces; and let $\vec{n}_i$ and $\vec{n}_j$ represent actual normals to the respective surfaces i and j at points p and q. Thus the necessary constraint equations are $\vec{r}_{pq} \times \vec{n}_i = 0$ and $\vec{r}_{pq} \times \vec{n}_j = 0$. This yields six constraint equations in four unknowns, which can be solved iteratively. The only requirement is that the resulting 4 by 4 coefficient matrix be positive definite, which it will be if $\left|\vec{r}_{pq}\right| > \varepsilon$, where ε is some small positive constant. When $\left|\vec{r}_{pq}\right| < \varepsilon$ the two surfaces are within ε of each other, and the solution will have been obtained.

Technically a 4 by 4 coefficient matrix is not required because Gaussian Elimination with Complete Pivoting could be applied to the 6 by 4 matrix, but setting up the 4 by 4 matrix is preferable for two reasons. First it is simpler to factor the 4 by 4 positive-definite matrix, and second the equivalent of a least-squares solution may be necessary when discretized surface representations are provided. In essence, there generally will be no discretized ϕ_i, θ_i, ϕ_j, and θ_j that exactly satisfy the six constraint equations. Thus we find the best ϕ_i, θ_i, ϕ_j, and θ_j that minimize the constraint violations.

Modeling the dynamic interaction between discretized surfaces is undesirable because it is difficult to compute reliable penetrations for generating contact forces and because artificial frequencies associated with the sharp angles between edges and points are generated in the dynamic simulations. Subdividing the local contact surfaces on the fly to match the resolution of analytical representations will minimize these problems. The Loop method of subdividing triangulated surfaces can be used to achieve any desired level of detail. When the surfaces at the (potential) contact regions have been sufficiently subdivided, the results will be equivalent to analytical representations.

The iterative GJK algorithm requires a local kth vertex matrix $\underline{r}_k^i = [r_x^i,\ r_y^i,\ r_z^i]^T$ of the ith hull, which is obtained from its .ply file. For any given (ϕ_i, θ_i) pair, the ith hull's

hash-table returns the associated face index. The corresponding face's vertex index information in the .ply file is then used to identify the kth vertex.

The iterative NRi algorithm is much more demanding than the GJK algorithm. It starts with the last face indices for hulls i and j identified by the GJK algorithm, but the arbitrary $\underline{r}_k^i$ and $\underline{r}_\ell^j$ vertex vectors on the corresponding faces may not be close to representing the actual points where (potential) contact takes place. However, they should be sufficiently close so that the NRi algorithm will converge. Taking $\underline{r}_k^i$ on hull i, identified by (ϕ_i, θ_i), and $\underline{r}_\ell^j$ on hull j, identified by (ϕ_j, θ_j), as initial starting vectors, we can make an initial update to ϕ_i, θ_i, ϕ_j, and θ_j using preliminary constraint equations and coefficient matrix.

The GJK algorithm had set the final normal vector according to its last search direction, represented here as $\underline{n}_{ij}^0 = [n_x^0,\ n_y^0,\ n_z^0]^T$. From $\underline{n}_{ij}^0$ was obtained local normal vectors $\underline{n}_i^i = [n_x^i,\ n_y^i,\ n_z^i]^T$ and $\underline{n}_j^j = [n_x^j,\ n_y^j,\ n_z^j]^T$. From $\underline{n}_i^i$ and $\underline{n}_j^j$ were obtained angle pairs (ϕ_i, θ_i) and (ϕ_j, θ_j) and local vertex vectors $\underline{r}_{ip}^i = [r_x^i,\ r_y^i,\ r_z^i]^T$ and $\underline{r}_{jq}^j = [r_x^j,\ r_y^j,\ r_z^j]^T$. From this we evaluate $\underline{r}_{pq}^0 = \underline{r}_{0j}^0 + \underline{R}^{0j}\underline{r}_{jq}^j - \underline{r}_{0i}^0 - \underline{R}^{0i}\underline{r}_{ip}^i$. Setting $\underline{n}_i^0 = \underline{R}^{0i}\underline{n}_i^i$ and $\underline{n}_j^0 = \underline{R}^{0j}\underline{n}_j^j$, the six constraint equations are $\tilde{\underline{r}}_{pq}^0\underline{n}_i^0 = 0$ and $\tilde{\underline{r}}_{pq}^0\underline{n}_j^0 = 0$. Evaluating the Jacobian matrix requires evaluating local partial derivatives of $\underline{n}_i^i$ and $\underline{r}_{ip}^i$ with respect to (ϕ_i, θ_i) and local partial derivatives of $\underline{n}_j^j$ and $\underline{r}_{jq}^j$ with respect to (ϕ_j, θ_j).

To see how the same information in a unit-sphere file may be applied to multiple convex hulls, consider that any convex hull may be mapped onto a unit sphere, where the unit sphere's face normal vectors may be arranged to best suit other requirements. The primary data in this file are Cartesian coordinates of the normal vectors $\underline{n}_k^i = [n_x^i,\ n_y^i,\ n_z^i]^T$ at the corresponding vertices in the first file and the definition of each triangle face. The data is stored in a .ply file where the $\underline{n}_k^i = [n_x^i,\ n_y^i,\ n_z^i]^T$ coordinates are actually the sphere's vertex coordinates because they are the same in a unit sphere. In addition, evaluating the coefficient matrix requires the first partial derivative of the normal vectors with respect to ϕ_i and θ_i. This yields two tangent vectors $\underline{t}_{\theta k}^i = [t_{\theta x}^i,\ t_{\theta y}^i,\ 0]^T$ and $\underline{t}_{\phi k}^i = [t_{\phi x}^i,\ t_{\phi y}^i,\ t_{\phi z}^i]^T$. Thus five additional quantities must be pre-computed and stored in the file. This extra data is stored in each row after the normal vectors, which are stored in the vertex vector locations. The quantities represented by $\underline{n}_k^i$, $\underline{t}_{\theta k}^i$, and $\underline{t}_{\phi k}^i$ are associated with the discretized unit sphere and remain the same for all convex hulls. Thus only the vertex locations represented by $\underline{r}_k^i$ vary from one convex hull to the next, which is

why the vertices are stored in separate files for each unique convex hull shape and size.

We will now show how Loop's surface subdivision algorithm works with the NRi algorithm to locate the exact minimum normal points p and q on the respective convex hull surfaces i and j. The continuous variables for surface i are θ_i and ϕ_i; likewise θ_j and ϕ_j for surface j. The NRi algorithm updates these four variables, and the required constraint equations and coefficient matrix must be updated with these updated variables.

We get smooth quantities from discretized surfaces as follows. First, the (θ_i, ϕ_i) and (θ_j, ϕ_j) pairs are each converted into a keyword, which are then inserted into a hash table to locate the closest vertex, normal, and tangent vectors to the desired points p and q. The hash table was generated from the discretized unit sphere, so the same table will apply to multiple convex hulls if all were mapped onto the same discretized unit sphere. Then the convex hull surfaces immediately surrounding the two identified vertices are subdivided a sufficient number of times to find points on the surfaces matching p and q, more specifically matching (θ_i, ϕ_i) and (θ_j, ϕ_j), to the desired resolution. The result of this subdivision is equivalent to analytical representations of the surfaces.

The subdivision method effectively divides triangles on the convex hulls into smaller triangles until new vertices fall close enough to the desired points p and q. But the subdivision process is controlled by manipulating the unit sphere, because we have no direct way of establishing the vertex locations p and q as functions of (θ_i, ϕ_i) and (θ_j, ϕ_j). To do this we compute ($\underline{n}_k^i$, $\underline{t}_{\theta k}^i$, $\underline{t}_{\phi k}^i$) and ($\underline{n}_k^j$, $\underline{t}_{\theta k}^j$, $\underline{t}_{\phi k}^j$) exactly because they are properties of the unit sphere, for which we know everything. Using values for ($\underline{n}_k^i$, $\underline{t}_{\theta k}^i$, $\underline{t}_{\phi k}^i$) and ($\underline{n}_k^j$, $\underline{t}_{\theta k}^j$, $\underline{t}_{\phi k}^j$) at the discrete vertices stored in the proper unit sphere files, we can linearly interpolate between the convex hull vertices to match the computed ($\underline{n}_k^i$, $\underline{t}_{\theta k}^i$, $\underline{t}_{\phi k}^i$) and ($\underline{n}_k^j$, $\underline{t}_{\theta k}^j$, $\underline{t}_{\phi k}^j$). In essence, coefficients generated in the sphere's linear interpolations will be the same as those used in the convex hull surface subdivisions to obtain the points p and q. Thus we are able to subdivide any arbitrary quantized convex hull the same way we would subdivide the unit sphere, which is trivial.

The main steps required to model contact between two arbitrary convex hulls are summarized next. Then each of these steps will be expanded and explained in sufficient detail to develop and understand the necessary algorithms.

1. On a unit sphere specify the desired minimum angular separation a in radians between adjacent normal vectors, or specify the number of vertices n. The surface area of a unit sphere is 4π steradians, so

$n = 4\pi / a^2$ is a good approximation of the number of vertices on the discretized unit sphere. A good rule of thumb for obtaining smooth surfaces is an angular difference between adjacent normal vectors of about 5° or 0.087 radians. This equates roughly to $n = 1650$ points. Mapping the surface of any convex hull onto the discretized surface of the unit sphere will give the same minimum angular separation a of its normal vectors.

2. Characterize the unit sphere in terms of two angles $0 \le \phi \le \pi$ and $0 \le \theta \le 2\pi$. On the unit sphere ϕ ranges from zero at the North Pole to π at the South Pole, and θ ranges from zero at the Prime Meridian, east toward 2π returning back to the Prime Meridian. The Cartesian coordinates $\underline{r}_k = [\cos\theta_k \sin\phi_k,\ \sin\theta_k \sin\phi_k,\ \cos\phi_k]^T$, which clearly are a unit vector, represent any point p_k on the surface of the unit sphere. The normal vector to the surface at this point is simply $\underline{n}_k = \underline{r}_k$. Select a set of values for ϕ_k and θ_k to give the desired minimum angular separation a between normal vectors as discussed in step 1. For each ϕ_k and θ_k store $\underline{n}_k = [\cos\theta_k \sin\phi_k,\ \sin\theta_k \sin\phi_k,\ \cos\phi_k]^T$, $\underline{t}^i_{\theta k} = [-\sin\theta_k \sin\phi_k,\ \cos\theta_k \sin\phi_k,\ 0]^T$, and $\underline{t}^i_{\phi k} = [\cos\theta_k \cos\phi_k,\ \sin\theta_k \cos\phi_k,\ -\sin\phi_k]^T$ in a special unit-sphere file. These quantities will apply to all convex hull models mapped onto this discretized unit sphere.
3. This step maps each convex hull onto the discretized unit sphere described above. A desired requirement is that the source convex hull vertex resolution be equal to or finer than the unit sphere resolution to get the most accurate results. If the convex hull surface quantization is too coarse, subdivision could be applied to obtain more representative vertices associated with the set of normal vectors established for the unit sphere. Unfortunately the vertices of arbitrary convex hulls don't always map uniformly onto the unit sphere because the same normal vector or very close ones may be associated with two or more vertices, sometimes widely spaced vertices if on large triangles or quads. When this happens it isn't clear which vertex to use. The best solution may be to use the weighted average of all vertices associated with closely aligned normal vectors. The set of representative vertices for the convex hull are saved in a separate vertex file in the same order as the information generated for the unit sphere in Step 2.
4. This step locates the index of the appropriate vertex generated and stored in the convex hull file from the previous step whose normal vector is most closely aligned with a search-direction vector $\vec{d}$. The algorithm normalizes $\vec{d}$ to $\underline{n} = [n_x,\ n_y, n_z]^T = [\cos\theta \sin\phi,\ \sin\theta \sin\phi,\ \cos\phi]^T$. From these coordinates it extracts ϕ and θ. Then it scales ϕ and θ upward, discretizes the scaled results, and generates a keyword for the hash table. From the hash table is obtained an index to the vertex in the convex hull

file and to related vector information in the unit sphere file as described above. The vertex from the convex hull file is basically all the GJK algorithm needs, but a modified version to speed up convergence may exploit some information from the unit sphere file as well.

5. This step applies the GJK algorithm to quickly zero in on the (potential) contact location between two convex hulls. All aspects of the GJK algorithm are described in detail later. A first estimate of the search direction, when a better one is not available, is the vector $\vec{r}_{ij}$ from the center of hull i to the center of hull j. The algorithm takes into account that the convex hulls are generally attached to translating and rotating bodies, so the direction vector's coordinates must be transformed to the local coordinate frames of hulls i and j. Without any enhancements to the GJK algorithm it will identify an index in hull i pointing to a vertex close to the desired point p, and a second index in hull j pointing to a vertex close to the desired point q. This will be the starting point for zeroing in on the actual points p and q, close to the minimal normal distance between the two surfaces.
6. Once the GJK algorithm has identified indices to vertices in the convex hull files at points p and q, the Newton-Raphson iteration algorithm, along with a surface subdivision process, will zero in on more accurate points p and q using higher resolution surfaces. The two normal vectors on hulls i and j, identified by the GJK algorithm, should be a good starting point for the NRi algorithm, so it should converge quadratically and fast. The NRi algorithm starts with the (θ_i, ϕ_i) and (θ_j, ϕ_j) variables returned from the GJK algorithm. It calls a function that returns a 4 by 4 positive-definite matrix and four constraint equations, which is described in the next paragraph. Then it solves for changes in the four variables and updates them. If the changes are small enough the NRi algorithm terminates, otherwise it performs another iteration.
7. Computing accurate surface penetration and reaction forces between two convex hulls generally requires a higher resolution surface model than the discretized version used by the GJK algorithm for fast contact detection. Subdividing surfaces is an advancing discipline used mostly in graphics rendering, but it is also being used more frequently in contact modeling as well. Subdividing surfaces is more than just inserting vertices on edges and faces, as it also pushes the new vertices out from the edges and faces to generate more rounded shapes to the surrounding surface. At the same time the new normal and tangent vectors take on values from the scaled average of those at the vertices surrounding it. The coefficient matrix used by the NRi algorithm doesn't have to be highly accurate, so it may not be necessary to update the tangent vectors when setting it up. The surface subdivision function works in conjunction with the NRi algorithm. Each time the function receives refined (θ_i, ϕ_i) and (θ_j, ϕ_j)

values, it performs an additional subdivision on the finer triangles surrounding the previous identified vertices if necessary.

A fast iterative method for computing the shortest normal distance between two convex hulls is summarized. Two arbitrary ellipsoids are used to illustrate and initially test the method, but any convex surface for which a constant potential function is available could be used. However the iterative algorithm does not rely on a constant potential function nor does it require smooth surfaces. Thus any two convex hulls represented by sets of vertices can also be used. This iterative algorithm is especially appealing when hash tables are used to return vertex locations associated with normal vectors. The potential of this iterative algorithm is that it may converge faster than the GJK algorithm. In addition, it automatically determines the center of the (potentially) contacting regions and their (separation) penetration.

For the purpose of illustrating the iterative contact detection algorithm, let the constant potential function

$$U_i = \frac{x_{ip}^{i2}}{a_i^2} + \frac{y_{ip}^{i2}}{b_i^2} + \frac{z_{ip}^{i2}}{c_i^2} = 1 \tag{1.1}$$

represent a typical ellipsoid i where a_i, b_i, and c_i are positive constants and x_{ip}^i, y_{ip}^i, and z_{ip}^i are its local surface point variables satisfying Eq. (1.1). The local surface point variables and x_{ip}^i, y_{ip}^i, and z_{ip}^i are measured relative to a local Cartesian coordinate frame, where its x, y, and z axes are aligned with the respective principal ellipsoid axes represented by the positive constants a_i, b_i, and c_i, and its origin is located at the geometrical center of the ellipsoid.

The gradient or potential field of Eq. (1.1) is

$$\nabla U_i = \begin{bmatrix} 2x_{ip}^i/a_i^2 & 2y_{ip}^i/b_i^2 & 2z_{ip}^i/c_i^2 \end{bmatrix} \tag{1.2}$$

which is normal to the ellipsoid's surface because it is the gradient of a constant potential. We can show that ∇U_i represents the surface normal vector using a parametric representation of the surface where

$$\begin{aligned} \underline{r}_{ip}^i &= \begin{bmatrix} x_{ip}^i & y_{ip}^i & z_{ip}^i \end{bmatrix}^T \\ &= \begin{bmatrix} a_i \cos\theta_i \sin\phi_i & b_i \sin\theta_i \sin\phi_i & c_i \cos\phi_i \end{bmatrix}^T \end{aligned} \tag{1.3}$$

It is easy to verify that the coordinates in Eq. (1.3) satisfy Eq. (1.1). In terms of the two parametric variables θ_i and ϕ_i, two tangent vectors to the surface are obtained as

$$\underline{t}^{\,i}_{\theta i} = \partial \underline{r}^{\,i}_{ip} / \partial \theta_i = \begin{bmatrix} -a_i \sin\theta_i \sin\phi_i & b_i \cos\theta_i \sin\phi_i & 0 \end{bmatrix}^T \tag{1.4}$$

and

$$\underline{t}^{\,i}_{\phi i} = \partial \underline{r}^{\,i}_{ip} / \partial \phi_i = \begin{bmatrix} a_i \cos\theta_i \cos\phi_i & b_i \sin\theta_i \cos\phi_i & -c_i \sin\phi_i \end{bmatrix}^T \tag{1.5}$$

Substituting Eq. (1.3) into Eq. (1.2) gives

$$\nabla U_i = \begin{bmatrix} 2\cos\theta_i \sin\phi_i / a_i & 2\sin\theta_i \sin\phi_i / b_i & 2\cos\phi_i / c_i \end{bmatrix} \tag{1.6}$$

It is easy to show that $\nabla U_i \bullet \underline{t}^{\,i}_{\theta i} = \nabla U_i \bullet \underline{t}^{\,i}_{\phi i} = 0$, which verifies that ∇U_i is normal to the surface. It is convenient to set the normal vector at any point p as

$$\underline{n}^{\,i}_{ip} = \nabla U_i^T / 2 = \begin{bmatrix} \cos\theta_i \sin\phi_i / a_i & \sin\theta_i \sin\phi_i / b_i & \cos\phi_i / c_i \end{bmatrix}^T \tag{1.7}$$

which is generally not a unit vector. While $\underline{n}^{\,i}_{ip}$ is not a unit vector, it is interesting that $\underline{n}^{\,iT}_{ip}\,\underline{r}^{\,i}_{ip} = 1$, which can be shown by combining Eqs. (1.3) and (1.7).

Suppose that

$$\underline{u}^{\,0}_{ij} = \begin{bmatrix} u^0_x & u^0_y & u^0_z \end{bmatrix}^T \tag{1.8}$$

represents the unit vector at the shortest normal distance between the two surfaces and is used to obtain the point $\underline{r}^{\,i}_{ip}$ on the ellipsoid, where $\underline{u}^{\,i}_{ip} = \underline{R}^{0iT}\underline{u}^{\,0}_{ij}$ is outward facing and orthogonal to the surface. Since $\underline{u}^{\,i}_{ip}$ and $\underline{n}^{\,i}_{ip}$ point in the same direction and are orthogonal to the surface at the same point, it follows that $\underline{u}^{\,i}_{ip}$ is a normalized representation of $\underline{n}^{\,i}_{ip}$. Noting that $\underline{u}^{\,i}_{ip}$ is normal to the ellipsoid surface at $\underline{r}^{\,i}_{ip}$, the variables θ_i and ϕ_i are computed from Eqs. (1.4) and (1.5), and $\underline{r}^{\,i}_{ip}$ is evaluated from Eq. (1.3). Taking the dot product of $\underline{u}^{\,i}_{ip}$ with $\underline{t}^{\,i}_{\theta i}$ in Eq. (1.4) and with $\underline{t}^{\,i}_{\phi i}$ in Eq. (1.5) gives

$$
\begin{aligned}
&-u_x^i a_i \sin\theta_i \sin\phi_i + u_y^i b_i \cos\theta_i \sin\phi_i = 0 \\
&u_x^i a_i \cos\theta_i \cos\phi_i + u_y^i b_i \sin\theta_i \cos\phi_i - u_z^i c_i \sin\phi_i = 0
\end{aligned}
\tag{1.9}
$$

From Eq. (1.9) is obtained

$$
\begin{aligned}
\theta_i &= \text{atan2}(u_y^i b_i, u_x^i a_i) \\
\phi_i &= \text{atan2}(u_x^i a_i \cos\theta_i + u_y^i b_i \sin\theta_i, u_z^i c_i)
\end{aligned}
\tag{1.10}
$$

and from Eq. (1.3) the desired point $\underline{r}_{ip}^i$ may be computed.

For a second ellipsoid j the process is similar. In this case the desired point q on the ellipsoid surface is represented by $\underline{r}_{jq}^j$ where $\underline{u}_{jq}^j = -\underline{R}^{0jT}\underline{u}_{ij}^0$ is outward facing and orthogonal to the surface. Since $\underline{u}_{jq}^j$ and $\underline{n}_{jq}^j$ point in the same direction and are orthogonal to the surface at the same point, it follows that $\underline{u}_{jq}^j$ is a normalized representation of $\underline{n}_{jq}^j$. Noting that $\underline{u}_{jq}^j$ is normal to the ellipsoid surface at $\underline{r}_{jq}^j$, the variables θ_j and ϕ_j are computed using equations similar to Eqs. (1.4) and (1.5), and $\underline{r}_{jq}^j$ is computed using an equation similar to Eq. (1.3). Taking the dot product of $\underline{u}_{jq}^j$ with the equivalent $\underline{t}_{\theta j}^j$ from Eq. (1.4) and with the equivalent $\underline{t}_{\phi j}^j$ from Eq. (1.5) gives the equivalent of Eq. (1.9) from which is obtained the equivalent of Eq. (1.10), and from the equivalent of Eq. (1.3) is obtained the desired point $\underline{r}_{jq}^j$.

The shortest normal distance between the two ellipsoids is represented as

$$
\underline{r}_{pq}^0 = \underline{r}_{0q}^0 - \underline{r}_{0p}^0 = \underline{r}_{0j}^0 + \underline{R}^{0j}\underline{r}_{jq}^j - \left(\underline{r}_{0i}^0 + \underline{R}^{0i}\underline{r}_{ip}^i\right)
\tag{1.11}
$$

between the point $\underline{r}_{ip}^i$ on ellipsoid i, obtained from $\underline{u}_{ip}^i = \underline{R}^{0iT}\underline{u}_{ij}^0$, and the point $\underline{r}_{jp}^j$ on ellipsoid j, obtained from $\underline{u}_{jq}^j = -\underline{R}^{0jT}\underline{u}_{ij}^0$. The coordinate matrices $\underline{r}_{0i}^0$ and $\underline{r}_{0j}^0$ represent the position of the Cartesian reference frame origins i and j relative to a fixed frame origin 0. Likewise the direction cosine matrices $\underline{R}^{0i}$ and $\underline{R}^{0j}$ represent the orientation of the Cartesian reference frames i and j relative to fixed frame 0.

Since $\underline{r}_{pq}^0$ represents the vector between points p and q on the respective surfaces where $\underline{u}_{ip}^i$ and $\underline{u}_{jq}^j$ are both parallel to $\underline{u}_{ij}^0$, it follows that these points lie on the same line defined by $\underline{u}_{ij}^0$ and so does $\underline{r}_{pq}^0$, which makes $\underline{r}_{pq}^0$ orthogonal to both surfaces and thus represents the minimum normal distance between them.

To start the iterative process, an exact value for $\underline{u}_{ij}^{0}$ is generally not available, so a best guess may be the best source. A prior value would be used when available. In the worst case of no knowledge, the first estimate for $\underline{u}_{ij}^{0}$ is obtained from

$$\underline{u}_{ij}^{0} = \left(\underline{r}_{0j}^{0} - \underline{r}_{0i}^{0}\right) \Big/ \left|\underline{r}_{0j}^{0} - \underline{r}_{0i}^{0}\right| \tag{1.12}$$

which would be exact if the ellipsoids were spheres. For convex hulls this starting vector insures that the points p and q will start out on portions of the two surfaces that face each other, which is the first requirement for convergence.

With $\underline{r}_{ip}^{i}$ and $\underline{r}_{jq}^{j}$ obtained from $\underline{u}_{ip}^{i}$ and $\underline{u}_{jq}^{j}$ based on $\underline{u}_{ij}^{0}$, $\underline{r}_{pq}^{0}$ is evaluated from Eq. (1.11). If $\underline{u}_{ij}^{0}$ is not the correct normal vector there will be a nonzero angle between $\underline{u}_{ij}^{0}$ and $\underline{r}_{pq}^{0}$, an indication that $\underline{u}_{ij}^{0}$ and thus $\underline{r}_{ip}^{i}$ and $\underline{r}_{jq}^{j}$ are not correct. To fix this alignment problem, unit vector $\underline{u}_{ij}^{0}$ and thus $\underline{u}_{ip}^{i}$ needs to be rotated toward the head of $\underline{r}_{pq}^{0}$ and simultaneously unit vector $\underline{u}_{jq}^{j}$ needs to be rotated toward its tail.

Determining how much to rotate $\underline{u}_{ij}^{0}$ and thus $\underline{u}_{ip}^{i}$ and $\underline{u}_{jq}^{j}$ is the crux of the next iteration. As $\underline{u}_{ip}^{i}$ is rotated toward the head of $\underline{r}_{pq}^{0}$ the tail of $\underline{r}_{pq}^{0}$ is moved along the surface of ellipsoid i toward the head of $\underline{r}_{pq}^{0}$ on the surface of ellipsoid j. In a similar manner as $\underline{u}_{jq}^{j}$ is rotated toward the tail of $\underline{r}_{pq}^{0}$ the head of $\underline{r}_{pq}^{0}$ is moved along the surface of ellipsoid j toward the tail of $\underline{r}_{pq}^{0}$ on the surface of ellipsoid i. The unit vectors $\underline{u}_{ip}^{i}$ and $\underline{u}_{jq}^{j}$ are rotated simultaneously with $\underline{u}_{ij}^{0}$ because they all represent the same line.

The amount $\underline{r}_{pq}^{0}$ is shortened on its head end depends on the local surface curvature of ellipsoid i, which is indicated by the length of $\underline{r}_{ip}^{i}$. In a similar manner the amount $\underline{r}_{pq}^{0}$ is shortened on its tail end depends on the local surface curvature of ellipsoid j, which is indicated by the length of $\underline{r}_{jq}^{j}$. Vector $\underline{r}_{pq}^{0}$ is at some oblique angle relative to unit vector $\underline{u}_{ij}^{0}$, so the combined tip displacement of $\underline{r}_{ip}^{0}$ and $\underline{r}_{jq}^{0}$ associated with their rotations would be approximately $\sqrt{\underline{r}_{pq}^{0T}\underline{r}_{pq}^{0} - \left(\underline{u}_{ij}^{0T}\underline{r}_{pq}^{0}\right)^{2}}$. This shows that when $\underline{r}_{pq}^{0}$ is parallel to $\underline{u}_{ij}^{0}$, the combined tip displacements associated with rotating $\underline{r}_{ip}^{0}$ and $\underline{r}_{jq}^{0}$ will be zero, which is the converged condition.

Let $\underline{r}_{ip}^{0}$ be rotated by angle α_i and $\underline{r}_{jq}^{0}$ by angle α_j where $|\alpha_i| = |\alpha_j|$. In essence we rotate both vectors by the same angle, but in opposite directions. In addition,

rotating the shorter vector will shorten $\underline{r}_{pq}^{0}$ less than rotating the longer vector. Thus we obtain

$$\left|\alpha_i\right| = \left|\alpha_j\right| = \frac{\sqrt{\underline{r}_{pq}^{0T}\underline{r}_{pq}^{0} - \left(\underline{u}_{ij}^{0T}\underline{r}_{pq}^{0}\right)^2}}{\left|\underline{r}_{ip}^{0}\right| + \left|\underline{r}_{jq}^{0}\right|} \tag{1.13}$$

where the proper signs on α_i and α_j are determined by which directions of rotating $\underline{r}_{ip}^{0}$ and $\underline{r}_{jq}^{0}$ are required to reduce the projection of $\underline{r}_{pq}^{0}$ orthogonal to $\underline{u}_{ij}^{0}$ to zero. Angle α_i is actually the amount $\underline{u}_{ij}^{0}$ must be rotated about an axis orthogonal to the plane containing the $\underline{r}_{ip}^{0}$ and $\underline{r}_{jq}^{0}$ vectors. The unit vector representing this rotation axis is obtained as

$$\underline{u}_{pq}^{0} = \frac{\tilde{\underline{r}}_{jq}^{0}\underline{r}_{ip}^{0}}{\left|\tilde{\underline{r}}_{jq}^{0}\underline{r}_{ip}^{0}\right|} \tag{1.14}$$

Using this unit vector and the angles α_i and α_j the two vectors would be rotated as

$$\begin{aligned} \underline{r}_{ip}^{0} &\leftarrow \cos\alpha_i\underline{r}_{ip}^{0} + \sin\alpha_i\tilde{\underline{u}}_{pq}^{0}\underline{r}_{ip}^{0} \\ \underline{r}_{jq}^{0} &\leftarrow \cos\alpha_j\underline{r}_{jq}^{0} + \sin\alpha_j\tilde{\underline{u}}_{pq}^{0}\underline{r}_{jq}^{0} \end{aligned} \tag{1.15}$$

However, it isn't the vectors $\underline{r}_{ip}^{0}$ and $\underline{r}_{jq}^{0}$ that need to be rotated but just $\underline{u}_{ij}^{0}$ itself. Thus $\underline{u}_{ij}^{0}$ is rotated as

$$\underline{u}_{ij}^{0} \leftarrow \cos\alpha_i\underline{u}_{ij}^{0} + \sin\alpha_i\tilde{\underline{u}}_{pq}^{0}\underline{u}_{ij}^{0} \tag{1.16}$$

With a new $\underline{u}_{ij}^{0}$ matrix we can set new $\underline{u}_{ip}^{i}$ and $\underline{u}_{jq}^{j}$ matrices, repeat Eqs. (1.10), (1.3), and (1.11) for hulls i and j, and rotate $\underline{u}_{ij}^{0}$ again until it is sufficiently parallel to $\underline{r}_{pq}^{0}$.

Next we transfer this iterative process to two arbitrary convex hulls i and j to determine the closest vertex points on their respective surfaces. For arbitrary convex hulls i and j with no constant potential functions we use hash tables to obtain $\underline{r}_{ip}^{i}$ given $\underline{u}_{ip}^{i}$ and $\underline{r}_{jq}^{j}$ given $\underline{u}_{jq}^{j}$. The hash tables associate the closest vertex $\underline{r}_{ip}^{i}$ or $\underline{r}_{jq}^{j}$ with the respective input unit normal vector $\underline{u}_{ip}^{i}$ or $\underline{u}_{jq}^{j}$. These are the same hash tables used in the GJK algorithm.

The hash tables make the iterative process even simpler because only Eq. (1.11) needs to be applied, combined with rotating $\underline{u}_{ij}^{0}$ until it is sufficiently parallel to $\underline{r}_{pq}^{0}$. Using hash-tables to evaluate $\underline{r}_{ip}^{i}$ and $\underline{r}_{jq}^{j}$ given $\underline{u}_{ip}^{i}$ and $\underline{u}_{jq}^{j}$ definitely makes this iterative algorithm a lot simpler than the GJK algorithm. In addition this algorithm automatically determines the contact location and surface separation or penetration. (It should be straightforward to add the equivalent of Eq. (1.11) to the GJK algorithm to get the same contact information.) Whether or not this algorithm is competitive with the GJK algorithm will likely depend on the number of iterations required to converge to the desired alignment of $\underline{u}_{ij}^{0}$ with $\underline{r}_{pq}^{0}$. Stability is another issue. Which algorithm is more likely to converge with different convex hull models and the worst-case starting conditions?

Working with discrete vertex points as opposed to smooth surfaces poses a problem for this algorithm that will require a modification of the convergence criteria. When surfaces are smooth and infinitely divisible, $\underline{r}_{pq}^{0}$ may take any orientation and be aligned as closely with $\underline{u}_{ij}^{0}$ as desired. But when discrete vertices represent the surface, $\underline{r}_{pq}^{0}$ may take only discrete orientations allowed by the discrete $\underline{r}_{ip}^{i}$ and $\underline{r}_{jq}^{j}$ vectors returned from the hash tables. This means that $\underline{r}_{pq}^{0}$ generally cannot represent the minimum-distance vector between the two convex hulls.

The most accurate method for obtaining $\underline{r}_{pq}^{0}$ would be to use some form of subdivision that produces a surface with resolution similar to the continuous functions. That is overkill for this first go-around, so Eq. (1.15) will be applied to approximate the $\underline{r}_{ip}^{i}$ and $\underline{r}_{jq}^{j}$ vectors. Then their lengths will be adjusted by linearly interpolating on the respective triangle that $\underline{r}_{ip}^{i}$ or $\underline{r}_{jq}^{j}$ intersects. This method essentially allows the closest vertex on each convex hull to penetrate nearby edges and faces on the other hull at some loss of accuracy. For initial testing purposes this should be adequate.

For this iterative algorithm all that needs to be stored between one time step and the next is the previous $\underline{u}_{ij}^{0}$, which immediately yields the starting $\underline{u}_{ip}^{i}$ and $\underline{u}_{jq}^{j}$, the starting $\underline{r}_{ip}^{i}$ and $\underline{r}_{jq}^{j}$ from hash-table lookups, and the starting $\underline{r}_{pq}^{0}$ from Eq. (1.11). Since generally little changes from one time step to the next, this algorithm should converge in one or two iterations. The GJK algorithm may also converge just as quickly when features from its last iteration are saved from the previous time step and used to start the iteration at the current time step.

Triangular B-Spline Representation of Convex Hull Surface

Triangular B-splines are used to achieve sufficient smoothness for modeling rolling and sliding contact between convex hull surfaces. The triangles are parameterized

locally with three Barycentric coordinate variables $0 \le u,v,w \le 1$, where $u+v+w=1$. These local parameters generally have little meaning to the surface contact evaluation function, which instead works with point locations, first and second time derivatives of the surface at the point locations, and normal vectors. The GJK algorithm generates direction vectors that must be aligned with face normal vectors. Hash tables are used to locate the triangles whose normal vectors are most closely aligned with the direction vectors. Once the GJK algorithm has converged, Newton-Raphson iteration is used to zero in on the closest normal points on the convex hull surfaces. A local set of Barycentric coordinates $0 \le u,v,w \le 1$ is used on each triangle, which will be discontinuous when moving from one triangle to the next. Thus these variables are unsuitable locating the closest normal points on the convex hull surfaces.